T3000 API & Protocol

Alarms · built Nov 13, 2026

Alarms

Alarm monitoring, configuration, and notification system.

Overview

The alarm system monitors critical conditions and notifies operators when values exceed limits or equipment fails.

Alarm Types

Value Alarms:

- High limit exceeded
- Low limit exceeded
- Rate of change alarm
- Deviation alarm

Status Alarms:

- Equipment failure
- Communication loss
- Sensor fault
- Override active

System Alarms:

- Database errors
- Server issues
- License expiration

Alarm Priorities

- 🔴 **High**: Critical safety/equipment issues
- 🟠 **Medium**: Important but not critical
- 🟡 **Low**: Information/warnings

- ◯ **Info**: Status changes

Alarm Page

View active and historical alarms:

Priority	Point	Message	Time	Status
High	Zone_Temp	High Temperature	10:23 AM	Active
Medium	Fan_Status	Feedback Error	10:15 AM	Acknowledged

Alarm Actions

- **Acknowledge**: Confirm awareness
- **Clear**: Resolve alarm condition
- **Silence**: Mute audio alerts
- **Filter**: View by priority/status

Notification Settings

Configure alarm notifications:

Email:

- Recipient addresses
- Subject line format
- Include alarm details

SMS:

- Phone numbers
- Message template
- Priority filtering

Browser:

- Desktop notifications
- Audio alerts
- Popup windows

Best Practices

- Set appropriate thresholds
- Use priority levels wisely

- Respond promptly
- Review alarm history
- Tune to reduce false alarms

Next Steps

- Inputs - Input alarms
- Monitoring - Real-time monitoring
- Troubleshooting - Resolve issues

Alarms

Alarm Management API

Alarm Data Structure

TYPESCRIPT

```typescript
interface Alarm {
  id: string;
  deviceId: number;
  pointId: string;
  type: 'high-limit' | 'low-limit' | 'fault' | 'override' | 'communication';
  priority: 'high' | 'medium' | 'low' | 'info';
  message: string;
  value: number;
  limit: number;
  timestamp: number;
  acknowledged: boolean;
  acknowledgedBy?: string;
  acknowledgedAt?: number;
}

// Get active alarms
const response = await fetch(`http://localhost:9103/api/alarms/active`);
const alarms: Alarm[] = await response.json();

// Acknowledge alarm
await fetch(`http://localhost:9103/api/alarms/${alarmId}/acknowledge`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ user: 'operator1' })
});
```

Alarm Detection Engine

TYPESCRIPT

```typescript
class AlarmEngine {
  private rules = new Map<string, AlarmRule>();
  private activeAlarms = new Map<string, Alarm>();

  checkPoint(pointId: string, value: number) {
    const rule = this.rules.get(pointId);
    if (!rule) return;

    const alarmId = `${pointId}:high-limit`;
    const isInAlarm = value > rule.highLimit;
    const existingAlarm = this.activeAlarms.get(alarmId);

    if (isInAlarm && !existingAlarm) {
      // New alarm
      if (this.checkDelay(pointId, rule.delay)) {
        this.triggerAlarm({
          id: alarmId,
          pointId,
          type: 'high-limit',
          priority: rule.priority,
          message: `${pointId} high limit exceeded`,
          value,
          limit: rule.highLimit
        });
      }
    } else if (!isInAlarm && existingAlarm) {
      // Alarm cleared
      this.clearAlarm(alarmId);
    }
  }

  private triggerAlarm(alarm: Alarm) {
    alarm.timestamp = Date.now();
    alarm.acknowledged = false;
    this.activeAlarms.set(alarm.id, alarm);

    // Send notifications
    this.notifySubscribers(alarm);
```

```
    this.sendEmail(alarm);
    this.logToDatabase(alarm);
  }
}
```

WebSocket Alarm Notifications

TYPESCRIPT

```typescript
// Subscribe to alarm events
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    channel: 'alarms',
    filters: {
      priority: ['high', 'medium'],
      deviceIds: [389001, 389002]
    }
  }));
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'alarm-triggered') {
    showNotification(data.alarm);
    playAlarmSound(data.alarm.priority);
  } else if (data.type === 'alarm-cleared') {
    clearNotification(data.alarmId);
  }
};
```

Email Notification

TYPESCRIPT

```typescript
import nodemailer from 'nodemailer';

class EmailNotifier {
  private transporter: nodemailer.Transporter;

  constructor(config: EmailConfig) {
    this.transporter = nodemailer.createTransporter({
      host: config.smtpHost,
      port: config.smtpPort,
      secure: config.useTLS,
      auth: {
        user: config.username,
        pass: config.password
      }
    });
  }

  async sendAlarmEmail(alarm: Alarm) {
    const html = `
      <h2>Alarm Notification</h2>
      <p><strong>Priority:</strong> ${alarm.priority.toUpperCase()}</p>
      <p><strong>Point:</strong> ${alarm.pointId}</p>
      <p><strong>Message:</strong> ${alarm.message}</p>
      <p><strong>Value:</strong> ${alarm.value}</p>
      <p><strong>Limit:</strong> ${alarm.limit}</p>
      <p><strong>Time:</strong> ${new Date(alarm.timestamp).toLocaleString()}</p>
    `;

    await this.transporter.sendMail({
      from: 'T3000 Alarms <alarms@t3000.local>',
      to: 'operator@example.com',
      subject: `[${alarm.priority.toUpperCase()}] ${alarm.pointId} ${alarm.type}`,
      html
    });
  }
}
```

Next Steps

- REST API
- WebSocket API
- Device Monitoring